HealthNexus
The ML explained

Features, the personal baseline, hyperparameters, how severity is decided, false-positive reduction, the pipeline and where the model lives

For reading and for defence: every number is from the code, the reports in model/reports, or a measurement made on this machine.
Generated 27 October 2026 from the live code. Every path:line link opens VS Code at that line.
Open any line by hand: Ctrl+P, paste the path and line, press Enter.

Contents

1. The big picture: what 'the ML' is in HealthNexus 3

2. Input features 4

3. The personal baseline: how it works 6

4. Constraints on the data, and the good answers 9

5. Hyperparameters 12

6. How severity (low, moderate, high, critical) is decided 15

7. Reducing false positives 18

8. Evaluation results 21

9. End-to-end workflow 24

10. Folder structure and where the model lives 26

11. Tech stack and response time 28

12. Defence Q&A 30

1 The big picture: what 'the ML' is in HealthNexus

Three pieces do machine-learning work. Knowing which one decides what is the first thing to say in a viva.

71	4,432	300	29 ms
people in the training data (LifeSnaps)	days of heart rate and breathing rate	trees in the Isolation Forest	to score one reading (median)

1.5%
false flags on normal days, was 24.6%

Piece	What it is	What it decides	Where it lives
Personal-baseline scorer (`personal-baseline-v1`)	Statistics: each person's own median and spread per vital, blended with published normal ranges	**Whether a reading is flagged, and how severe** (low, moderate, high, critical)	`app/services/scoring.py:239`
Population model (`population-iforest-v1`)	A trained **Isolation Forest** (scikit-learn) on heart rate and breathing rate	**Nothing on its own.** It adds a 'population check' beside each reading: how unusual next to 71 people	`app/services/population.py:69`
Outbreak clustering	Space-time **DBSCAN** plus a Poisson test	Whether several people near each other have the same illness more than usual	`model/healthnexus_ml/clustering.py:79`

> **Say this first:** *the Isolation Forest does not decide low, moderate, high or critical.* Those levels come from how far a vital is outside its normal range or the person's own baseline (chapter 6). The forest only reports a percentile, and the research showed it added no detection beyond the rules, so it is shown as information and never raises a flag.

Why two layers instead of one model

- **There are no labels.** Nobody marked which days were 'ill'. A supervised classifier cannot be trained; the options are unsupervised models (Isolation Forest) or statistics that explain themselves.
- **People differ more than days do.** For resting heart rate, HRV and breathing rate, people's averages differ about **3 times more than one person changes day to day** (notebook 02). A model trained on everyone mostly learns 'who is this person', not 'is today unusual'. A personal baseline is therefore needed.
- **The Isolation Forest alone barely beats guessing** at spotting a personal change (ROC AUC 0.52 to 0.61), while personal baselines reached 0.90.
- **A flag must explain itself.** A worker is told 'oxygen saturation 89%, well below the normal range', which a forest score cannot say.

What the PDF covers

Input features (2), the personal baseline (3), data constraints and good answers (4), hyperparameters (5), how severity is decided (6), false-positive reduction (7), results (8), the end-to-end workflow (9), the folder structure and where the model lives (10), tech stack and response time (11), and a defence Q&A (12).

2 Input features

Three different sets of inputs exist: what the research trained on, what the population model uses, and what the live scorer reads.

2.1 Training data: LifeSnaps

LifeSnaps (Yfantidou et al., 2022, *Scientific Data*, CC BY 4.0): Fitbit data from **71 people over about four months in 2021**. The raw daily file has 63 columns; ten were kept because a normal smartwatch also gives them.

Signal (our name)	Raw column	Used in personal score?	Why
resting_hr	resting_hr	**yes**	Rises with infection; maps to the app's `heart_rate`
hrv_rmssd	rmssd	**yes**	Falls when the body is stressed or ill
sleep_hr	nremhr	**yes**	Heart rate in deep sleep
breathing_rate	full_sleep_breathing_rate	**yes**	Maps to the app's `respiratory_rate`
skin_temp	nightly_temperature	**yes**	Temperature trend at night
sleep_minutes, sleep_efficiency	minutesAsleep, sleep_efficiency	no	Context only
steps, very_active_minutes, sedentary_minutes	steps, very_active_minutes, sedentary_minutes	no	Depends on what someone chose to do (gym, lazy Sunday), not on health. Using them made **25% of normal days flag**.

Left out on purpose: `spo2` (only 17% of days have it), `stress_score` (itself a model output, on 25% of days), `calories`, `bpm` and `distance` (duplicates of steps and heart rate), and age, gender and body measurements.

2.2 The population model uses two inputs

Model input	From LifeSnaps	Matches the app's
heart_rate	resting heart rate	`heart_rate` (bpm)
respiratory_rate	breathing rate during sleep	`respiratory_rate` (/min)

These are the only two LifeSnaps signals that match something a citizen's app measures, so the ten-signal wearable model could not be used on app readings. The forest is trained on exactly these two (`vitals.py`), and a missing one is treated as 'the population median' (nothing unusual).

2.3 The live scorer reads eight vitals

Vital	Unit	Normal band	Plausible range (typo guard)	Safety limit (always counts)	Smallest spread allowed
`systolic`	mmHg	90 to 120	50 to 300	at or below 85 · at or above 160	5
`diastolic`	mmHg	60 to 80	30 to 200	at or below 50 · at or above 100	4

Vital	Unit	Normal band	Plausible range (typo guard)	Safety limit (always counts)	Smallest spread allowed
`heart_rate`	bpm	60 to 100	25 to 250	at or below 45 · at or above 120	4
`temperature_c`	°C	36.1 to 37.2	30 to 45	at or below 35.5 · at or above 38	0.2
`spo2`	%	95 to 100	50 to 100	at or below 92	1
`respiratory_rate`	/min	12 to 20	4 to 60	at or below 9 · at or above 24	1.5
`glucose_mg_dl`	mg/dL	70 to 99	20 to 600	at or below 65 · at or above 180	8
`weight_kg`	kg	none	2 to 400	none	0.8

File	Role
app/services/metrics.py:21	The vitals vocabulary: label, unit, normal band, plausible range.
app/services/scoring.py:12	Safety limits per vital.
app/services/scoring.py:87	Floors for a person's spread.

2.4 Where the readings come from

- **Smartwatch sync** (four a day, simulated in this build): heart rate, SpO_2, breathing rate, temperature.
- **A health worker's examination:** blood pressure, glucose, weight and any of the others.
- Each reading is one row of `health_records.vitals` (a JSON object) and is scored the moment it is stored.

2.5 Symptoms are not model inputs

The 12 symptoms a person can record (fever, cough, headache, body ache, breathlessness, chest pain, dizziness, fatigue, nausea, diarrhoea, thirst, rash) **do not change the anomaly score**. They are shown to the worker and are used by the **outbreak clustering**, which maps them to three syndromes (febrile, respiratory, gastrointestinal) in `features.py`.

Questions you may be asked

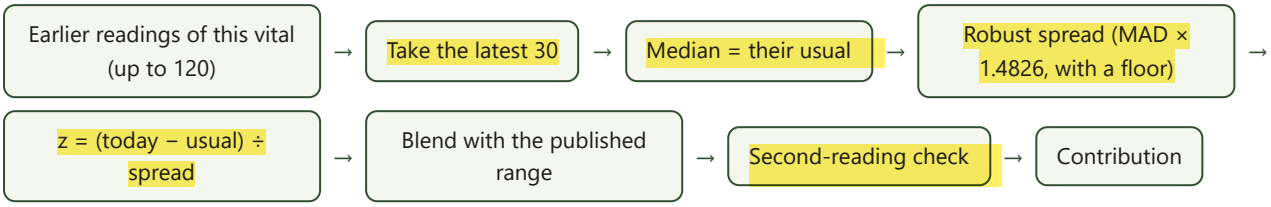
Q. Why did you not use all ten signals in the personal score?
Steps and activity depend on choices, not health. In the first try, taking the biggest z-score over all ten signals flagged about 25% of normal days, almost always because of steps. Only the five body signals are used.

Q. Why only two features for the Isolation Forest?
Those are the only two that exist both in the research data and in what the app measures. A forest trained on signals the app never has would be useless at run time.

Q. Do symptoms affect severity?
No. Severity depends only on vitals. Symptoms inform the worker and the outbreak detector.

3 The personal baseline: how it works

Each person is compared with their own usual values, once there are enough of them, and with published normal ranges before that.



3.1 The four ingredients

Ingredient	Formula	Why
Usual value	median of the person's latest 30 readings of that vital, **earlier ones only**	The median is not pulled by one odd reading; excluding today stops an abnormal reading being compared with itself.
Spread	`max(1.4826 × MAD, smallest-allowed spread, 1e-6)`	MAD is the median absolute deviation (robust to outliers); 1.4826 makes it comparable to a standard deviation; the floor stops a very regular person getting a near-zero spread and being flagged for ordinary noise.
z-score	(today − usual) ÷ spread	How many 'normal wobbles' away today is.
Personal contribution	`max(0, |z| − 1.5) ÷ 1.5`	Nothing until 1.5 spreads away; then each further 1.5 adds 1.

3.2 From published ranges to personal: the weight

Per vital, the weight moves gradually from the published normal band to the personal baseline as readings accumulate, so there is **no sudden jump** when a baseline becomes available.

Earlier readings of that vital	Stage	Weight on personal	Judged by
3	learning	0.00	100% published range / 0% personal
6	learning	0.00	100% published range / 0% personal
7	blending	0.07	93% published range / 7% personal
10	blending	0.27	73% published range / 27% personal
14	blending	0.53	47% published range / 53% personal
20	blending	0.93	7% published range / 93% personal
21	personal	1.00	0% published range / 100% personal
30	personal	1.00	0% published range / 100% personal

```
contribution = (1 − weight) × published_contribution + weight × personal_contribution
published_contribution = distance outside the normal band, in half-band widths (0 inside the band)
```

File	Role
app/services/scoring.py:163	0 below 7 readings, rising to 1 at 21.
app/services/scoring.py:177	Median and floored robust spread of the latest 30.
app/services/scoring.py:186	`max(0, |z| − 1.5) ÷ 1.5`.
app/services/scoring.py:144	Distance outside the normal band in half-band widths.
app/services/readings.py:52	Loads up to 120 earlier readings, oldest first, for the baseline.

3.3 The confirmation rule (one odd reading is usually noise)

A value that is **inside the published range but unusual for this person** counts only if the **previous** reading of that vital was also outside their usual band, in the same direction. Until then it is reported as 'watch' and contributes nothing. Illness keeps a value off for days; a hurried measurement does not.

	Reading	Baseline (median, spread)	z-score	Personal part	Confirmed?	Contribution	Flagged?
Day 1	heart rate 78	62, 4	+4.0	1.67	no (needs a second reading)	0.00	no, only 'watch'
Day 2	heart rate 79	62, 4	+4.2	1.83	yes (day 1 was also unusual, same direction)	1.83	yes

3.4 Safety limits override the baseline

A value past a **safety limit** (for example SpO_2 at or below 92, temperature at or above 38.0 °C, systolic at or above 160) always counts in full, whatever the person's usual: a fever is a fever. The limits are wider than the normal bands so that someone with controlled chronic hypertension (say 145/92) is judged against their own baseline instead of being flagged on every reading.

3.5 From contributions to a flag

- A vital counts only if its contribution is at least **0.5** (`FLAG_THRESHOLD`); safety limits count in full.
- `anomaly_score = min(1, strongest contribution ÷ 4)`. A reading is flagged when any vital counts.
- `basis` records whether the flag used published ranges or a personal baseline; the page tells the citizen and worker which.
- Everything is stored with the score (`processed_data.details`), so any flag can be explained later.

Function / component	Where	What it does
`_judge`	app/services/scoring.py:198	One vital: published part, personal part, weight, safety and the confirmation check.
`_contribution`	app/services/scoring.py:229	Blends the two parts; applies the 0.5 threshold and the safety rule.
`score_reading`	app/services/scoring.py:239	The only function the rest of the app calls: returns score, flag, features, basis, details, watch list and population check.

Questions you may be asked

Q. Why median and MAD instead of mean and standard deviation?

A single outlier moves a mean and standard deviation a lot, which would hide the next unusual value. The median and MAD barely move.

Q. What happens to a brand-new citizen?

Fewer than 7 readings of a vital means weight 0: only the published normal range is used, so they are not left unprotected. The dashboard says how many readings remain until a personal baseline starts.

Q. Why must the baseline exclude today's reading?

Otherwise an abnormal reading would be partly compared with itself and look less abnormal.

4 Constraints on the data, and the good answers

Every rule that limits what data the model sees, with the reason for it. Each reason is the answer to give if asked 'why?'.

4.1 Cleaning the research data (preprocessing.py)

Constraint	Value	The good answer (why)
Drop 'non-wear' days	1440 sedentary minutes **and** no steps **and** no resting heart rate	Fitbit records exactly 1440 when the watch is off. Left in, the model would learn that '0 steps' is normal and miss real drops.
Impossible values become missing (not deleted)	resting HR 30 to 120 · HRV 5 to 200 · sleeping HR 30 to 120 · breathing 6 to 30 · skin temp 30 to 38 · steps 0 to 60,000	Sensors glitch. The rest of the day may still be fine, so only the value is blanked. The ranges are wide **on purpose**: they catch broken sensors, not real unusual readings, which are exactly what the model must find.
Sleep under 2 hours clears the overnight signals	`sleep_minutes` below 120 → HRV, sleep HR, breathing, skin temp become missing	A nap or a recording that cut out is not comparable with a full night.
1440 sedentary minutes on a kept day means no data	set to missing	It is not a lazy day; it is the watch being off.
Per-person history counters	`day_index`, and for each signal a running count of valid values, **up to and including** that day	Lets the model know how much history exists on each day, with **no look into the future** (no leakage).
Split by **person**, not by day	56 people to train, 15 to test	If days were split, the model would already have seen the test people's other days and look better than it is for a brand-new user.
Personal baseline uses earlier days only	`shift(1)` then a rolling window	The day being scored must not be part of its own baseline.
Robust standardising	population median and MAD × 1.4826; missing → 0	The data has outliers. Missing → 0 means 'same as the median person', because a forest cannot take empty values and half the days have no HRV.

File	Role
model/healthnexus_ml/preprocessing.py:35	The plausible ranges.
model/healthnexus_ml/preprocessing.py:66	Non-wear rule.
model/healthnexus_ml/preprocessing.py:73	Impossible values and the short-sleep rule.
model/healthnexus_ml/preprocessing.py:86	The per-person counters.

4.2 What the app accepts as input

Constraint	Value	The good answer (why)
Plausible range per vital	e.g. SpO_2 50 to 100, heart rate 25 to 250, temperature 30 to 45 °C	Outside it a value is a **typo, not an anomaly**: the form says 'check for a typo' instead of raising a false alarm.

Constraint	Value	The good answer (why)
Unknown measurements rejected; at least one required	`VitalsIn` validator	Nothing the scorer does not understand can be stored.
Systolic must be above diastolic	`_pressure_order`	A swapped pair is a data-entry error.
Symptoms	at most 12, duplicates removed	Keeps a record readable and bounded.
Time of measurement	not in the future (5 minutes' tolerance), at most 30 days back	A reading from long ago would distort the baseline order; the real storage time is kept separately in `created_at`.
Examination needs a place	2 to 160 characters	A worker's measurement must say where it was taken.
Health-tracking consent	off → no reading is stored	A person who turned tracking off must not have readings created for them.
Only the citizen's **own** history	`earlier_vitals` filters on the user id	A baseline must be personal; another person's data never enters.
History available to the scorer	latest 120 readings; baseline from the latest 30; at least 7 to start	Bounds the work per reading; below 7 readings a baseline would be a guess, so none is used ('no baseline is better than a fake one').
Watch simulator limits	heart rate 40 to 180, SpO_2 85 to 100, breathing 8 to 35, temperature 34.5 to 41	Simulated values stay physiologically possible.

File	Role
`app/schemas/health.py:33`	Request rules for every reading.
`app/services/metrics.py:55`	The plausible-range check.
`app/services/watch.py:50`	Simulator clamps.

4.3 Honest limits of the data

- **No labels.** Nobody marked which days were illness, so there is no accuracy on real illness; the episodes used for testing are **synthetic** (a rise of heart rate and breathing rate injected into real days).
- **Resting values stand in for spot readings.** LifeSnaps has a calm once-a-day heart rate; real spot readings are noisier, so the false-flag rates here are a **lower bound**.
- **Only heart rate and breathing rate could be tested** on real data; blood pressure, SpO_2, temperature and glucose use the same code but have no real test data.
- **71 people, 15 of them held out**, so every number comes from a small sample.
- **Glucose's normal band is the fasting range (70 to 99)**, so a normal after-meal 130 mg/dL scores as 'high'. A meal-aware band is future work.

Questions you may be asked

Q. Why not delete rows with impossible values?

Because the same day's other signals may be fine. Setting only the broken value to missing keeps the rest of the evidence.

Q. How do you know the model works if there are no labels?
Three ways: injected illness-like episodes at three strengths, false flags on ordinary days, and (as a negative result) mood check-ins, which showed no link. The limits are stated, not hidden.

5 Hyperparameters

Every tunable value, where it is defined, and what was done to choose it.

5.1 The Isolation Forest (population model)

Hyperparameter	Value	Meaning and why
`n_estimators`	300	Number of random trees. More trees give a steadier score; 300 is well past the point where the score stops changing.
`contamination`	0.03	The share of training days expected to be unusual (3%). It sets the cut that turns the raw score into 'score 0.5 = more unusual than 97% of training days'.
`max_samples`	auto, which is 256	Each tree is built from 256 random days (scikit-learn's default).
`random_state`	42	Makes training repeatable.
Standardising	median and MAD × 1.4826	Robust to outliers; missing values become 0 ('same as the median person').
Score scaling	percentile 0.94 → 0, about 0.97 → 0.5, 0.995 → 1	`score = clip((percentile − (1 − 2c)) ÷ ((1 − c/6) − (1 − 2c)))` with c = contamination. 'Unusual' means score 0.5 or more.
Training data	4,432 days from 71 people	Two features: heart rate and breathing rate.

File	Role
`model/healthnexus_ml/anomaly.py:54`	Where the forest is built (`class GlobalModel`).
`model/healthnexus_ml/anomaly.py:73`	The percentile-to-score scaling.
`model/artifacts/vitals_anomaly.json:1`	The model card: versions, medians, spreads.

5.2 The personal-baseline scorer

Constant	Value	Meaning	How it was chosen
`MIN_BASELINE_READINGS`	7	Readings before any personal baseline	About a week of daily readings; fewer is a guess
`FULL_BASELINE_READINGS`	21	Readings for a fully personal baseline	Three weeks. Ablation: changing it to 14 or 30 barely changed results
`BASELINE_WINDOW`	30	Latest readings used for the median and spread	Recent enough to follow a person's drift
`SCORING_HISTORY`	120	Earlier readings loaded per scoring	Bounds the work
`PERSONAL_OK_Z`	1.5	z beyond which a personal change starts to count	Tuned: the first version (2.5) caught only 16% of moderate episodes; 1.5 catches 68%

Constant	Value	Meaning	How it was chosen
`PERSONAL_UNIT_Z`	1.5	Each further z of this adds 1 to the contribution	Paired with the line above
`FLAG_THRESHOLD`	0.5	A vital counts only from this contribution	The knee of the trade-off curve (figure in chapter 8)
`SCORE_CEILING`	4.0	A contribution of this size is score 1.0	Spreads severities across 0 to 1
`MAD_TO_SD`	1.4826	Makes MAD comparable to a standard deviation	A statistical constant
Spread floors	systolic 5, diastolic 4, heart_rate 4, temperature_c 0.2, spo2 1, respiratory_rate 1.5, glucose_mg_dl 8, weight_kg 0.8	Smallest spread a baseline may have	About the repeatability of home devices. Without floors false flags rose 6 times

Severity band	From anomaly_score
critical	0.75
high	0.5
moderate	0.25
low	0.0

File	Role
`app/services/scoring.py:45`	All the constants above.
`app/services/triage.py:18`	The severity bands.

5.3 Outbreak clustering (DBSCAN and the Poisson test)

Parameter	Value	Meaning
`radius_km` (DBSCAN eps)	1.5	How close cases must be
`km_per_day`	0.5	One day apart counts as this many kilometres, so time is a third axis
`min_people` (min_samples)	3	Smallest cluster; three is also the smallest group any area figure in the app shows
Window / baseline	14 days / 56 days	Current cases against the same area's earlier level
Significance	p below 0.01	Poisson tail: unlikely to be chance
Expected-cases floor	0.5	Stops a tiny baseline making any cluster look extreme

5.4 Research-only values (notebooks 02 and 03)

`WINDOW` 60 days, `MIN_DAYS` 7, `FULL_DAYS` 21, `FLAG_THRESHOLD` 0.5 and `ALERT_DAYS` 2 (alert only after two flagged days in a row) in `anomaly.py` belong to the **wearable research model**. The app uses its own constants above.

5.5 How the values were chosen

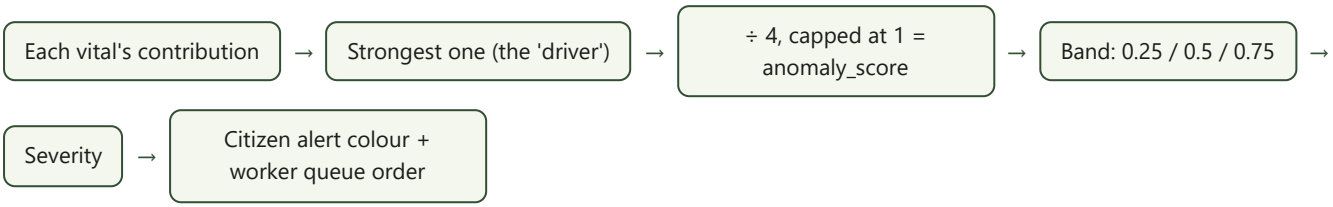
Notebook 04 ran a small grid over the two settings that matter most. Result (false flags on normal days, share of 3-day episodes caught for people with history):

PERSONAL_OK_Z	FLAG_THRESHOLD	False flags	Mild	Moderate	Strong
1.5	0.25	3.7%	37%	87%	100%
1.5	0.5	1.5%	16%	68%	98%
2.0	0.25	3.1%	8%	60%	98%
2.0	0.5	1.0%	3%	45%	97%
2.5	0.25	2.7%	2%	35%	94%
2.5	0.5	0.9%	2%	16%	76%

> **Chosen: 1.5 and 0.5.** 1.5 with threshold 0.25 catches more (87% of moderate) but more than doubles false flags (3.7%), which is too many alerts for a health worker.

6 How severity (low, moderate, high, critical) is decided

Severity is the size of the strongest contribution, not an Isolation Forest output. This chapter shows the arithmetic on real inputs.



anomaly_score	Severity	Meaning	Citizen's alert
below 0.25	low	flagged, but only just past the threshold	low (green)
0.25 to 0.50	moderate	clearly outside range	medium (amber)
0.50 to 0.75	high	well outside range	high (red)
0.75 and above	critical	far outside range	high (red)

6.1 The arithmetic

- **Published range part:** how far outside the normal band the value is, in **half-band widths**. Example: SpO_2 normal band 95 to 100 has a half-band of 2.5; a reading of 89 is 6 below, which is 2.4 half-bands.
- **Personal part** (once there are readings): `max(0, |z| − 1.5) ÷ 1.5`, blended in by the weight (chapter 3).
- **Safety limits** raise a value to at least 0.5 so it always flags.
- The reading's score is `strongest contribution ÷ 4` capped at 1, then compared with the bands. The vital that produced the strongest contribution is the **driver** and gives the sentence the user reads.

6.2 Worked examples (computed live from the real scorer)

A new person's reading	Strongest contribution	anomaly_score = min(1, c ÷ 4)	Severity	Citizen's alert level
Oxygen saturation 96%	below 0.5	0	not flagged	none
Oxygen saturation 92%	1.20	0.300	**moderate**	medium
Oxygen saturation 89%	2.40	0.600	**high**	high
Oxygen saturation 85%	4.00	1.000	**critical**	high
Temperature 37.5 °C	0.55	0.138	**low**	low
Temperature 38.0 °C	1.45	0.362	**moderate**	medium
Temperature 38.4 °C	2.18	0.545	**high**	high
Temperature 39.3 °C	3.82	0.955	**critical**	high
Heart rate 105 bpm	below 0.5	0	not flagged	none
Heart rate 125 bpm	1.25	0.312	**moderate**	medium
Heart rate 150 bpm	2.50	0.625	**high**	high

A new person's reading	Strongest contribution	anomaly_score = min(1, c ÷ 4)	Severity	Citizen's alert level
Blood pressure 130/85	0.67	0.168	**low**	low
Blood pressure 145/92	1.67	0.417	**moderate**	medium
Blood pressure 167/103	3.13	0.782	**critical**	high
Blood pressure 190/120	4.67	1.000	**critical**	high
Breathing rate 22 /min	0.50	0.125	**low**	low
Breathing rate 27 /min	1.75	0.438	**moderate**	medium

A person with history is judged the same way but against **their own** usual: a heart rate of 78 is 'critical-sized' for someone whose usual is 55 with a tight spread, and nothing for someone whose usual is 75. That is the point of the personal baseline.

File	Role
`app/services/triage.py:22`	Score to band.
`app/services/triage.py:66`	Builds 'Oxygen saturation 89% — well below the normal range'.
`app/services/triage.py:36`	Four bands to the alerts table's three levels.
`app/api/routes/phc.py:102`	The worker dashboard's SQL uses the same bands, so counts and cards agree.

6.3 What the Isolation Forest says, and what it does not

For every reading that has heart rate or breathing rate the forest returns a **percentile**: the share of the 4,432 training days that were *less* unusual than this one. A score of 0.5 or more is labelled 'unusual' and shown to the citizen and worker as a **population check**.

Example	Input	Percentile of the 4,432 training days	Population score	'Unusual' (score 0.5 or more)
typical resting person	heart rate 68, respiratory rate 15	34.4%	0.00	no
fast and breathing hard	heart rate 112, respiratory rate 24	100.0%	1.00	yes
very fast and breathing very hard	heart rate 150, respiratory rate 32	100.0%	1.00	yes
heart rate only, 58 bpm	heart rate 58	39.2%	0.00	no

> **It never sets severity and never raises a flag.** `evaluate.py` tried letting it flag new users at three thresholds (1.0, 0.75, 0.5): on 15 held-out people it added **no** detection, because most readings it calls unusual are ordinary for that person. Letting it flag would only add false alarms.

Questions you may be asked

Q. So how does the Isolation Forest give low, moderate, high and critical?

It does not. Those four levels come from the size of the strongest vital contribution (distance outside the range or the person's baseline). The forest provides a separate percentile-based population check shown beside the reading.

Q. Why show the population check at all if it cannot flag?

It is honest extra context: 'next to 71 people's data this heart rate and breathing rate are unusual / typical'. A worker may find it useful; it never alarms a citizen on its own.

Q. Why divide by 4?

A contribution of 4 is four half-bands outside the range (for example SpO_2 85), which is clearly critical. Dividing by 4 spreads real readings across the 0 to 1 scale so the four bands are all reachable.

7 Reducing false positives

Yes, a lot was done, and each step was measured. This chapter lists every measure, the number that justified it, and what was not done.

24.6%	1.5%	15 → 0	~3×
false flags before (published ranges only)	false flags now	people flagged on more than half their days	fewer false alarms from the second-reading rule

A **false positive** here is a flagged reading from a person who is simply well. It matters because a health worker who sees too many will stop reading alerts, and a citizen who is alarmed for nothing stops trusting the app.

7.1 Every measure, with its evidence

#	Measure	Evidence	Code
1	**Personal baseline instead of published ranges alone**	The old placeholder flagged **24.6% of normal days** and 15 of 71 people on more than half their days (fit people with a resting heart rate in the 50s, healthy but below '60 to 100'). Now 1.5% and nobody.	app/services/scoring.py:186
2	**A vital counts only from contribution 0.5** (`FLAG_THRESHOLD`)	Without it, a value one unit outside the band still flagged while the baseline was blending in: **13 of 20 healthy demo citizens** were flagged. Sweep: 0 → 10.1%, 0.25 → 3.7%, **0.5 → 1.5%**, 0.75 → 0.65%.	app/services/scoring.py:78
3	**Second unusual reading needed** for a change that is only unusual for that person	With realistic one-off spikes (+12 to +20 bpm on 5% of readings): **5.1% false flags without the rule, 1.5% with it**. Cost: moderate episodes caught 74% → 68%, and detection on day 2 instead of day 1.	app/services/scoring.py:213
4	**Smallest allowed spread** per vital	Without the floors, false flags rose **6 times** (9.1%): a very regular person's 'normal band' gets so narrow that everyday noise trips it.	app/services/scoring.py:87
5	**Gradual blend** from published to personal (7 to 21 readings)	No sudden change in behaviour; false flags by stage: learning 3.5%, blending 0.75%, personal 1.4%.	app/services/scoring.py:163
6	**Safety limits wider than the normal bands**	A person with controlled chronic hypertension (say 145/92) is judged against their own baseline instead of flagged on every reading.	app/services/scoring.py:12
7	**Median and MAD** (robust statistics); baseline excludes the reading being judged	One odd value neither inflates the spread nor hides the next unusual one.	app/services/scoring.py:177
8	**Typos blocked before scoring** (plausible ranges)	A mistyped 1200 for 120 is rejected with 'check for a typo' instead of becoming a critical flag.	app/services/metrics.py:55
9	**Activity signals left out** of the research personal score	Including steps flagged about 25% of normal days.	model/healthnexus_ml/anomaly.py:17

#	Measure	Evidence	Code
10	**The population forest may not flag**	Letting it flag new users at 1.0, 0.75 or 0.5 added no detection on 15 held-out people; most 'unusual' readings were ordinary for the person.	`app/services/population.py:69`
11	**Alert only after two flagged days in a row** (research alert rule)	About **3 times fewer** notifications on normal data; cost: moderate 3-day episodes caught 90% → 53%.	`model/healthnexus_ml/anomaly.py:23`
12	**A human reviews every flag**, grouped per person, with Resolve or Dismiss **and a reason**	A worker clears a person's flags in one action; the reason (false alarm, already treated, contacted, other) is stored on the flag.	`app/api/routes/phc.py:374`
13	**Honest wording to the citizen**	A flag is shown as 'a prompt to check, not a diagnosis', and a citizen is told plainly when nobody is reviewing it.	`app/services/readings.py:63`

7.2 What was not done (say this before you are asked)

- **No feedback loop yet.** A dismissal 'false alarm' is stored but is not used to retrain or tune thresholds. That is the natural next step: use dismissal reasons as weak labels.
- **No per-person threshold tuning.** All citizens share the same constants; only their baselines differ.
- **No validation on real illness.** Detection rates come from synthetic episodes; false-flag rates come from days assumed normal.
- **New people are the weakest case:** with fewer than 7 readings only published ranges apply, so 3.5% of readings flag (still far below the old 24.6%).

7.3 The ablation table (what each design choice earns)

Variant (one change at a time)	False flags on normal days	Moderate episodes caught	Mild caught
app model (as shipped)	1.5%	68%	16%
no second-reading rule	1.7%	74%	21%
personal band 2.5 SD (first version)	0.9%	16%	2%
full baseline at 14 readings	1.4%	68%	16%
full baseline at 30 readings	1.5%	66%	16%
no spread floor	9.1%	98%	79%
old placeholder	24.6%	16%	19%

> Read it as: **no spread floor** detects more (98%) but flags 6 times more people for nothing; the **2.5 SD first version** flags almost nobody but also misses most episodes; the **old placeholder** is worst on false flags and its detection is mostly luck. The shipped model is the balance.

Questions you may be asked

Q. Which single change reduced false positives the most?

Moving from published ranges alone to personal baselines: from 24.6% to 1.5% of normal days, and from 15 people flagged daily to none.

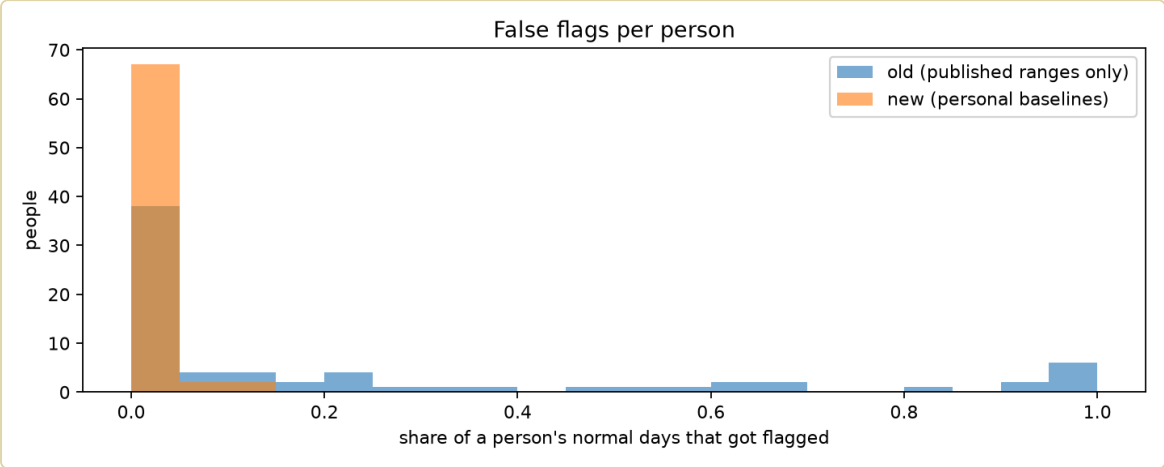
Q. Does the second-reading rule delay a real alert?

By one reading: most detections happen on day 2 of an episode. For something that develops over days that was judged acceptable; a safety-limit value (a real fever, low SpO_2) flags immediately.

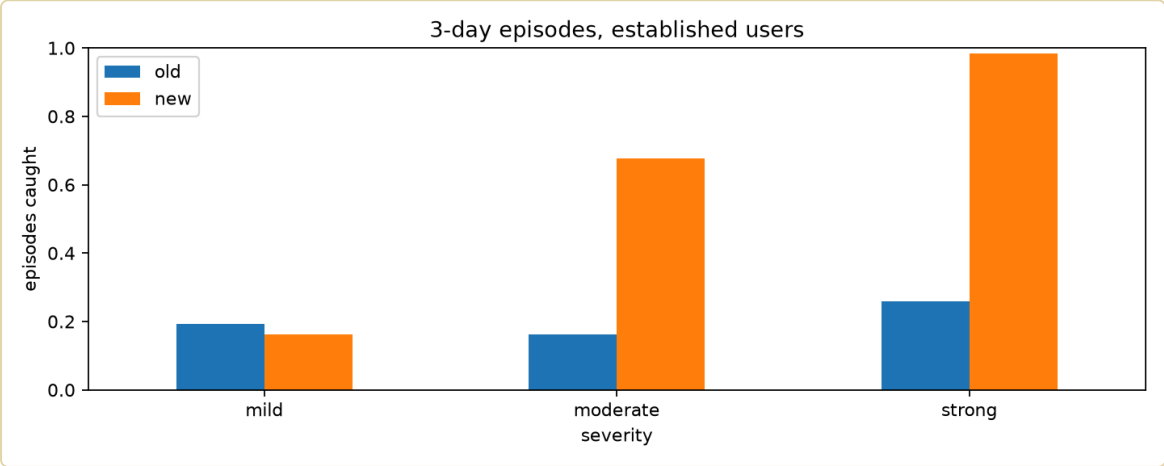
8 Evaluation results

Measured on the 71-person LifeSnaps data with the app's own scorer imported from the backend, so these are the real app's numbers.

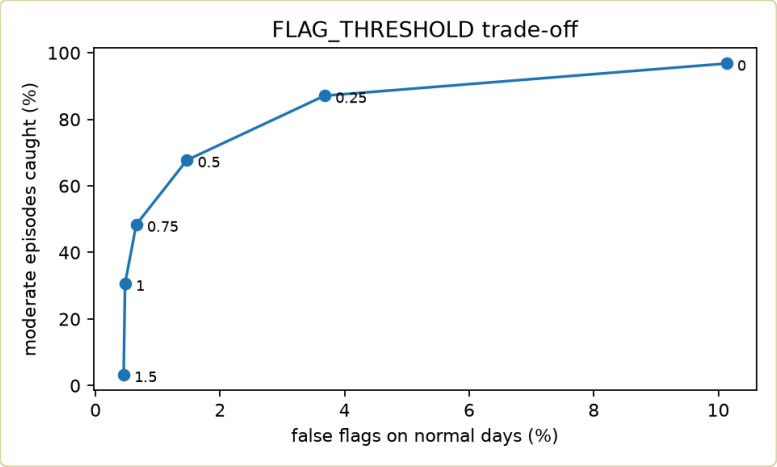
	Old placeholder	New scorer
False flags on normal days	24.6%	1.5%
People flagged on more than half their days	15 of 71	0
Moderate episodes caught (people with history)	16%	68%
Strong episodes caught (people with history)	26%	98%
Moderate episodes caught (brand-new users)	18%	57%
Mild episodes caught	19%	16% (accepted: the size of normal day-to-day change)



False flags per person. Blue is the old rule; orange is the personal baseline: almost everyone is near zero.



Three-day illness-like episodes caught, people with history. Strong and moderate episodes are caught far more often.



Choosing FLAG_THRESHOLD: 0.5 is the knee of the curve. Lower flags more normal days; higher misses episodes quickly.

8.1 How the episodes were made

There are no illness labels, so **three-day episodes were injected** into real days: heart rate and breathing rate pushed up together (mild +6 bpm and +1.5 breaths, moderate +10 and +2.5, strong +15 and +4), after the person's 25th reading for established users and from their 4th for new users. Most of the pushes stay **inside** the normal range (for example 58 → 68 bpm), which is exactly what published ranges cannot see.

8.2 The threshold sweep

FLAG_THRESHOLD	False flags	Moderate episodes caught
0.0	10.13%	97%
0.25	3.68%	87%
0.5	1.47%	68%
0.75	0.65%	48%
1.0	0.47%	31%
1.5	0.45%	3%

8.3 The population model, tested separately

Measure	Value
Test people / readings (held out)	15 / 1249
Readings the forest calls unusual	4.3%
...of which the app also flagged	13%
App flags the forest agrees with	47%
Detection added by letting the forest flag new users	none, at all three thresholds

Decision recorded in the report: *the app shows the population check with every reading but does not let it flag.*

8.4 Other findings reported

- Global (population) Isolation Forest alone: ROC AUC only **0.52 to 0.61** at catching a personal change; personal baselines reached **0.90**.

- Detection grows with history: 0% on global-only days, 22% while blending, **58%** once fully personal (notebook 02).
- **Negative result:** flagged days showed no link to self-reported tense or tired moods (blended score 0.088 on bad-mood days against 0.096 on others). The model cannot be claimed to detect stress.

9 End-to-end workflow

Two pipelines: the offline one that builds and checks the model, and the online one that scores every reading.

9.1 Offline: from data to a model the app can load

LifeSnaps raw CSV → preprocessing → daily_clean.csv → train → vitals_anomaly.joblib + model card → evaluate → reports/*.json + figures → restart the API

#	Layer	What happens	Open in code
1	Model	Load the raw daily Fitbit CSV (63 columns), keep ten, rename them.	model/healthnexus_ml/preprocessing.py:56
2	Model	Drop non-wear days, blank impossible values, clear short-sleep nights, add the per-person history counters, save `daily_clean.csv`.	model/healthnexus_ml/preprocessing.py:97
3	Model	Keep the two signals that match app vitals (resting HR to heart_rate, breathing rate to respiratory_rate).	model/healthnexus_ml/vitals.py:26
4	Model	Fit the forest on the median and MAD standardised data; keep the sorted training scores for percentiles.	model/healthnexus_ml/anomaly.py:49
5	Model	Save a plain dict (features, medians, scales, forest, training scores, contamination, version) with `joblib` and write the model card.	model/healthnexus_ml/train.py:51
6	Model	Evaluate: the app's real scorer is imported from the backend; the forest is refitted on 56 people and tested on 15.	model/healthnexus_ml/evaluate.py:26
7	Model	Write the metrics and figures into `model/reports`.	model/healthnexus_ml/evaluate.py:131

```
cd model
python -m healthnexus_ml.preprocessing   # raw CSV -> data/processed/daily_clean.csv
python -m healthnexus_ml.train            # -> artifacts/
python -m healthnexus_ml.evaluate         # -> reports/*.json (a few minutes)
python -m pytest tests
# then restart the API: it loads the model once
```

9.2 Online: what happens to one reading

#	Layer	What happens	Open in code
1	Router	A reading arrives from the smartwatch sync or a worker's examination and is validated.	app/schemas/health.py:33
2	Service	The citizen's earlier readings (up to 120, oldest first) are loaded.	app/services/readings.py:94
3	Database	The reading is stored in `health_records`.	app/services/readings.py:95

#	Layer	What happens	Open in code
4	Model	`score_reading` judges each vital: published part, personal part, weight, safety, confirmation; returns score, flag, features and details.	app/services/scoring.py:239
5	Model	`population.check` runs the Isolation Forest on heart rate and breathing rate (the artifact is loaded once and cached).	app/services/population.py:69
6	Database	`processed_data` stores score, features, per-vital working and the population view, so a flag can explain itself later.	app/services/readings.py:114
7	Service	A flagged reading gets its sentence (`describe_signal`) and a severity (`severity_of`), and an `Alert` for the citizen.	app/services/triage.py:66
8	Email	Emails are queued for the citizen and, if the flag can be reviewed, the centre's workers, and sent only after the database commit.	app/services/notify.py:68
9	Browser	The citizen's dashboard, history and alerts, and the worker's worklist, read the stored score and its explanation.	src/components/app/ModelInsights.tsx:32

9.3 Tests that guard the pipeline

File	What it checks
`model/tests/test_preprocessing.py`	Non-wear days dropped, impossible values blanked, short sleep clears overnight signals, history counters are per person and cumulative
`model/tests/test_anomaly.py`	The weight ramp, the baseline never uses today's value, flags run across gaps and people, a change normal for the population is still caught personally, normal days rarely flag, the saved model scores the same after reloading
`model/tests/test_clustering.py`	A tight group is found, one person repeating is not a cluster, the same place weeks apart is not one cluster, the Poisson tail, syndromes
`backend/tests/test_scoring.py`	The app scorer's rules

10 Folder structure and where the model lives

The training code and the model file are in `model/`; the app code that uses them is in `backend/app/services`. The two meet at one file.

10.1 The folders (line counts and sizes are real)

```
model/                              everything about the anomaly models
  healthnexus_ml/                   the Python package
    preprocessing.py          116   cleans the LifeSnaps daily CSV
    anomaly.py                162   the Isolation Forest class; research baselines
    vitals.py                  34   the app's model: the two shared signals
    train.py                   66   trains the models, writes artifacts/
    evaluate.py               179   evaluates the scorer and forest, writes reports/
    features.py                22   symptoms and vitals to syndromes
    clustering.py             117   space-time DBSCAN and the Poisson test
  artifacts/                        trained files
    vitals_anomaly.joblib   987 KB  THE MODEL THE API LOADS (in git)
    vitals_anomaly.json             its model card
    global_anomaly.joblib  3,032 KB  ten-signal research model (not in git)
  notebooks/                        01 to 04: preprocessing, model, alert rule, scorer
  reports/                          metrics JSON files and figures/*.png
  data/                             LifeSnaps files and processed CSV (not in git)
  tests/                            preprocessing, anomaly, clustering tests
  pyproject.toml, uv.lock           the package's environment (Python 3.12)
backend/app/services/               where the app uses it
    scoring.py                268   THE SCORER: baselines and the flag decision
    population.py             114   loads the forest, answers population checks
    triage.py                  88   severity bands and the plain-language signal
    readings.py               159   stores a reading, scores it, writes the alert
    metrics.py                 82   vitals vocabulary: bands, plausible ranges
    outbreaks.py              150   runs the clustering every 6 hours
```

10.2 Where the model lives, step by step

Question	Answer
Which file is the trained model?	`model/artifacts/vitals_anomaly.joblib` (987 KB). It is the only trained file tracked in git.
What is inside it?	A plain dictionary: the two feature names, the population medians and scales, the **scikit-learn** `IsolationForest` (300 trees), the sorted training scores (for percentiles), the contamination, the version and training date. No custom classes, so the backend can load it without the model package.
Who loads it?	app/services/population.py:45: `joblib.load`, cached with `lru_cache`, loaded **once** on the first reading after the server starts.
From which path?	app/services/population.py:21 (the repository's `model/artifacts` folder), or the `POPULATION_MODEL_PATH` setting if set.
Who calls it?	app/services/scoring.py:266: the last line of `score_reading`.
Where is its answer kept?	In `processed_data.population` (JSON) beside the score, and shown by the page as the 'population check'.
What if the file is missing or will not load?	The API logs it once and carries on: readings are scored by the rules and simply have no population check.

Question	Answer
How is it rebuilt?	`python -m healthnexus_ml.train` (needs the LifeSnaps files), then restart the API.

> **One sentence for the examiner:** *the model is a 1 MB scikit-learn file in `model/artifacts`, loaded by `backend/app/services/population.py` and called from `scoring.py` for every reading; the rules in `scoring.py` decide the flag.*

10.3 What each file does

File	Role
`model/healthnexus_ml/preprocessing.py:97`	Raw CSV in, clean per-person daily table out.
`model/healthnexus_ml/anomaly.py:37`	The forest, its standardising, percentile and score; `to_bundle` and `from_bundle`.
`model/healthnexus_ml/anomaly.py:121`	The research scorer (global and personal blended, 2-day alert rule): used by the notebooks, not the app.
`model/healthnexus_ml/vitals.py:22`	The two signals shared with the app and the model version.
`model/healthnexus_ml/train.py:40`	Trains both models and writes the model card.
`model/healthnexus_ml/evaluate.py:131`	The evaluation that produced the report files.
`model/healthnexus_ml/features.py:3`	Symptoms to the three syndromes used by outbreak detection.
`model/healthnexus_ml/clustering.py:79`	DBSCAN over space and time, and `poisson_tail`.
`app/services/scoring.py:239`	The only function the app calls to judge a reading.
`app/services/population.py:69`	Percentile and score for a reading's heart rate and breathing rate.

Scratch files: `model/data/processed/sample.csv`, `new.py` and similar files are your preprocessing experiments. Nothing in the pipeline reads them.

Environments: the model package targets Python 3.12 (`model/.venv`); the API runs in its own environment with the **same scikit-learn version** (1.9.1) so the saved forest loads.

11 Tech stack and response time

What the ML side is built with, and how fast it is, measured on this machine.

Layer	Technology	Used for
Language	Python 3.12 (model package), the API on its own Python	All ML code
Numerics and data	numpy 2, pandas 2.2	Cleaning, standardising, the research scoring
Machine learning	scikit-learn 1.9.1: `IsolationForest`, `DBSCAN`	Population model; outbreak clustering
Model storage	joblib (compressed dict)	`vitals_anomaly.joblib`
Research	JupyterLab, matplotlib	Notebooks 01 to 04 and the figures
Packaging	hatchling, uv (`pyproject.toml`, `uv.lock`)	Installing `healthnexus_ml` (also installed into the backend with `-e ../model`)
Tests	pytest	Preprocessing, anomaly model, clustering
Serving	FastAPI + SQLAlchemy async + PostgreSQL	Scoring runs inside the request that stores the reading
Data	LifeSnaps (CC BY 4.0), 71 people	Training and evaluation

11.1 Response time (measured)

27.6 ms
score one reading, median (30 earlier readings)

28.9 ms
with 120 earlier readings

0.1 ms
the rules alone

0.38 s
load the model file (once)

986 KB
model file size

Step	Median	95th percentile	Note
`score_reading` with the forest	27.6 ms	40.2 ms	30 earlier readings; the whole scoring of one reading
`score_reading` with 120 earlier readings	28.9 ms	43.7 ms	The most history the app ever supplies
`population.check` alone	29.3 ms	45.2 ms	Isolation Forest on one row of 300 trees
`score_reading`, rules only	0.1 ms	0.2 ms	Personal baseline, blend, safety, confirmation
Import scikit-learn and joblib	3.3 s		Once per server start
`joblib.load` of the model	0.38 s		Once per server start (cached)

- Measured on a developer laptop running Windows, one request at a time, scikit-learn 1.9.1, 300 timings per row, on 2026-10-27.
- **The forest is about 99.5% of the scoring time.** The rules take about a tenth of a millisecond; scikit-learn's per-call overhead for predicting **one row** across 300 trees is the rest. It is still negligible next to a database round trip and a

network request.

- **The first reading after a server start is slower**: importing scikit-learn (about 3 s) and loading the model (under 1 s), once. In one cold measurement it took about 15 s. A start-up warm-up call would remove this from the first user's request (an easy improvement, not yet done).
- Nothing here is a bottleneck for four smartwatch readings a day per citizen; scoring happens synchronously in the request that stores the reading.

Questions you may be asked

Q. Is 29 ms fast enough?

Yes. A reading is stored and scored in one request, a person sends a few a day, and 29 ms is small next to the database and network time. If volume grew, the first step would be scoring several readings per call or dropping the forest from the hot path, since it never decides a flag.

Q. Why is the first call slow?

The scikit-learn import and the model load are lazy (`lru_cache`), so they happen on the first reading after a start. A warm-up call at start-up would hide it.

12 Defence Q&A

Short answers to the questions a technical examiner is most likely to ask about the ML.

Questions you may be asked

Q. Is this really machine learning, or just rules?

Both, and each is used where it fits. There is a trained, unsupervised model (an Isolation Forest on 4,432 days from 71 people) and a statistical learning layer that estimates each person's own baseline from their own data. The flag decision is deliberately the explainable layer, because a health worker needs 'oxygen saturation 89%, well below the normal range', not a score.

Q. Why an Isolation Forest?

There are no labels, so the model has to be unsupervised. It handles several signals together, is fast, needs no distribution assumption, and isolates unusual points in fewer random splits.

Q. Why not a deep model such as an LSTM?

71 people and four months of data with no labels is far too little, and a deep model would not be able to explain a flag. The research showed the simple personal baseline already reaches AUC 0.90 on injected episodes.

Q. How can you claim it works with no labelled illness?

I do not claim accuracy on real illness. I measured two things honestly: false flags on ordinary days (1.5%, down from 24.6%) and detection of **injected** illness-like episodes (98% of strong, 68% of moderate for people with history). The limits are written down.

Q. Were the thresholds tuned on the test data?

The forest was tested on 15 people it never saw. The scorer's constants (1.5 and 0.5) were chosen with a grid over the same 71-person data, so the reported numbers are somewhat optimistic. A truly held-out set, or real worker feedback, is the next step.

Q. Why does the Isolation Forest not flag?

I tested it. At three thresholds it added no detection on 15 held-out people, because a population model mostly learns who a person is, and most readings it calls unusual are ordinary for that person. It is shown as context only.

Q. What does contamination 0.03 mean?

I expect about 3% of days to be unusual. It sets the cut for 'score 0.5 means more unusual than 97% of training days'.

Q. How is a brand-new user protected?

With fewer than 7 readings only the published normal range applies, so a clearly abnormal reading still flags; 3.5% of normal readings flag in that stage, against 24.6% before.

Q. Does the model learn from users' data?

The forest is trained once and never updated by users. Each person's baseline is computed on demand from their own readings and is not shared, so no one's data trains a model another person is scored by.

Q. What happens if the model file is missing?

Scoring continues with the rules; readings just carry no population check. This is tested by design: the check returns `None`.

Q. Why pin scikit-learn at 1.9.1?

A saved forest is a pickled object; loading it in a different scikit-learn version can fail or change behaviour. The model card records the version it was trained with.

Q. What would you improve first?

Use workers' 'false alarm' dismissals as weak labels to tune thresholds; validate on real wearable and spot-reading data; a meal-aware glucose band; warm the model up at start-up; per-vital tuning where real data exists.

Q. What is the privacy position?

Training data is the public, anonymised LifeSnaps set (CC BY 4.0). No citizen's readings are used to train anything. Area-level outbreak views count only people who opted in, and never a group under three.